

Requirements Analysis

Project Lead: Aiden Cary (Project and Backend Lead)

Team Members: Keller Willhite (UI/UX Developer), Zachery Atchley (Integration and Unit Tester)

Date: March 3, 2026

1. Introduction & Problem Domain

CodePulse is a code quality and bug prediction dashboard designed to verify if source code adheres to established standards and provides feedback on potential vulnerabilities. By utilizing a **Repository Architecture**, the system transforms raw code into a central **ASTObject** (Abstract Syntax Tree) that allows multiple specialized engines to perform simultaneous analysis. This approach addresses the need for real-time, actionable feedback during the development process.

2. Stakeholders & User Classes

Understanding the participants is the first step in defining the system's problem domain:

- **Individual Developers (Primary):** Users who interact with the dashboard for real-time code analysis, refactoring suggestions, and style compliance.
 - **Team Leads (Favored Users):** Users requiring high-security encryption for proprietary code and advanced reporting to track technical debt.
 - **System Administrators:** Responsible for managing the **Supabase/PostgreSQL** database, user authentication policies, and monitoring API health.
-

3. Functional Requirements (The "What")

These define the specific behaviors and services the system must exhibit to provide value.

- **REQ-1 (Web Dashboard):** The system shall provide a JavaScript-based web dashboard for users to paste and submit source code.
- **REQ-2 (Authentication):** The system shall implement secure user accounts, storing a `password_hash` to protect personal submission history.
- **REQ-3 (Code Encryption):** The system shall encrypt all user-submitted source code before storage in the PostgreSQL database.

- **REQ-4 (Structural Parsing):** The system shall use a **CodeParser** to generate an **ASTObject** representing the code's hierarchy and symbol table.
 - **REQ-5 (Static Analysis):** The **StaticEngine** shall calculate cyclomatic complexity and identify dead code or bloated classes.
 - **REQ-6 (Standards Compliance):** The system shall detect deviations from the **PEP 8** coding standard, including naming conventions and indentation.
 - **REQ-7 (AI-Assisted Prediction):** The system shall utilize an AI model (e.g., CodeBERT) to identify risky patterns and predict potential bugs based on historical data.
 - **REQ-8 (Actionable Reporting):** The system shall generate a "Quality Report" containing **Findings** with line numbers, severity levels, and specific suggested fixes.
 - **REQ-9 (History Management):** The system shall allow users to retrieve and review past code submissions and their associated analysis results.
-

4. Non-Functional Requirements (The "How Well")

These define the constraints and quality targets of the CodePulse system.

ID	Category	Specification
REQ-10	Efficiency	The system shall produce and display analysis results within 5 seconds of the user's submission.
REQ-11	Learnability	A new user shall be able to complete their first full code analysis within 2 minutes of their initial login.
REQ-12	Confidentiality	The system shall implement Row Level Security (RLS) in Supabase to ensure users only access their own data.

REQ-13	Reliability	The system shall handle codebases of up to 5,000 lines without performance degradation or crashes.
REQ-14	Usability	The dashboard must remain interactive and display a loading state while backend analysis is in progress.

5. Technical Constraints & Environment

- **Frontend:** JavaScript/React-based web application deployed via **Vercel**.
- **Backend:** **FastAPI (Python)** orchestrating the static analysis and AI prediction modules.
- **Database:** **Supabase (PostgreSQL)** for authentication, metadata, and encrypted persistence.
- **Standards:** All Python code must follow **PEP 8**; JS/React components must use **PascalCase** naming.